

API Reference

ETSI GS QKD 004 push mode specification, QKD server

API Version: 1.0.0

Server description of the QKD endpoint processing ETSI GS QKD 004 push mode requests

CONTACT

URL: <https://qkd-kms.ait.ac.at>

INDEX

1. ETSI004	3
1.1 POST /open-connect	3
1.2 GET /get-key	4
1.3 PUT /close	4

API

1. ETSI004

1.1 POST /open-connect

open key stream

The open-connect message connects a KMS to the QKD Device. Conforming to the specification, the QKD device will assign the key_stream_id and can suggest alternative QoS due to its capabilities.

REQUEST

REQUEST BODY - application/json

```
{
  source*           string           the source URI of the request.
  destination*      string           the destination URI of the request.
  key_stream_id     string           key stream ID. If not specified it is up to the QKD device to define the UUID
  qos* {
    All values according to ETSI GS QKD 004 specification. If KMS does not want to specify a parameter, it is up to the choice of the QKD.
    Either the value is given as "null" or not present.
    key_chunk_size  integer          length of key chunks in Byte.
    max_bps         integer          in bit per second
    min_bps         integer          in bit per second
    jitter          integer          in bit per second
    priority        integer          Meaning up to implementation.
    timeout         integer          Timeout in msec after which the call will be aborted, returning an error.
    ttl            integer          Time in seconds after which the keys for this key stream ID shall be erased from the kms key store.
    metadata_mimetype string        MIME type of the metadata of the `get` request
  }
}
```

RESPONSE

STATUS CODE - 201: Key stream successfully created

RESPONSE MODEL - application/json

```
{
  source*           string           the source URI of the request.
  destination*      string           the destination URI of the request.
  key_stream_id     string           key stream ID. If not specified it is up to the QKD device to define the UUID
  qos* {
    All values according to ETSI GS QKD 004 specification. If KMS does not want to specify a parameter, it is up to the choice of the QKD.
    Either the value is given as "null" or not present.
    key_chunk_size  integer          length of key chunks in Byte.
    max_bps         integer          in bit per second
    min_bps         integer          in bit per second
    jitter          integer          in bit per second
    priority        integer          Meaning up to implementation.
    timeout         integer          Timeout in msec after which the call will be aborted, returning an error.
    ttl            integer          Time in seconds after which the keys for this key stream ID shall be erased from the kms key store.
    metadata_mimetype string        MIME type of the metadata of the `get` request
  }
  status            enum            ALLOWED:0, 1, 2, 3, 4, 5, 6, 7, 8
                                Status code:
                                0 = Successful
                                1 = Successful connection, but peer not connected
                                2 = GET_KEY failed because insufficient key available
                                3 = GET_KEY failed because peer application is not yet connected
                                4 = No QKD connection available
                                5 = OPEN_CONNECT failed because the KSID is already in use
                                6 = TIMEOUT_ERROR: The call failed because the specified TIMEOUT
                                7 = OPEN failed because requested QoS settings could not be met, counter proposal
                                included in return has occurred
                                8 = GET_KEY failed because metadata field size insufficient. Returned Metadata_size
}
```

```
}
```

value holds minimum needed size of metadata

STATUS CODE - 400: Client sent a malformed input

STATUS CODE - 500: The server experienced an internal error

1.2 GET /get-key

start pushed key retrieval process

The get-key message is the starting point after which the QKD device can start pushing keys.

REQUEST

REQUEST BODY - application/json

```
{
  key_stream_id* string corresponding to the one given in `open_connect`
  index integer increasing index of the key chunk in the key stream. Wrapping at UINT_MAX.
  metadata {
    size integer size of buffer in characters
    buffer string metadata in json format.
  }
}
```

RESPONSE

STATUS CODE - 205: Successful registration

RESPONSE MODEL - application/json

```
{
  status* enum ALLOWED:0, 1, 2, 3, 4, 5, 6, 7, 8
  Status code:
  0 = Successful
  1 = Successful connection, but peer not connected
  2 = GET_KEY failed because insufficient key available
  3 = GET_KEY failed because peer application is not yet connected
  4 = No QKD connection available
  5 = OPEN_CONNECT failed because the KSID is already in use
  6 = TIMEOUT_ERROR: The call failed because the specified TIMEOUT
  7 = OPEN failed because requested QoS settings could not be met, counter proposal included in return has occurred
  8 = GET_KEY failed because metadata field size insufficient. Returned Metadata_size value holds minimum needed
  size of metadata
  metadata {
    size integer size of buffer in characters
    buffer string metadata in json format.
  }
}
```

STATUS CODE - 400: Client sent a malformed input

STATUS CODE - 500: The server experienced an internal error

1.3 PUT /close

close the key stream

The close message disconnects the KMS from the QKD device.

REQUEST

REQUEST BODY - application/json

```
{
  key_stream_id* string corresponding to the one given in `open_connect`
}
```

RESPONSE

STATUS CODE - 204: Key stream successfully closed

RESPONSE MODEL - application/json

```
{
  status* enum ALLOWED:0, 1, 2, 3, 4, 5, 6, 7, 8
    Status code:
    0 = Successful
    1 = Successful connection, but peer not connected
    2 = GET_KEY failed because insufficient key available
    3 = GET_KEY failed because peer application is not yet connected
    4 = No QKD connection available
    5 = OPEN_CONNECT failed because the KSID is already in use
    6 = TIMEOUT_ERROR: The call failed because the specified TIMEOUT
    7 = OPEN failed because requested QoS settings could not be met, counter proposal included in return has occurred
    8 = GET_KEY failed because metadata field size insufficient. Returned Metadata_size value holds minimum needed
    size of metadata
}
```

STATUS CODE - 400: Client sent a malformed input

STATUS CODE - 500: The server experienced an internal error
